长程工作流：Harness、Loop 与 Graph

张虎来

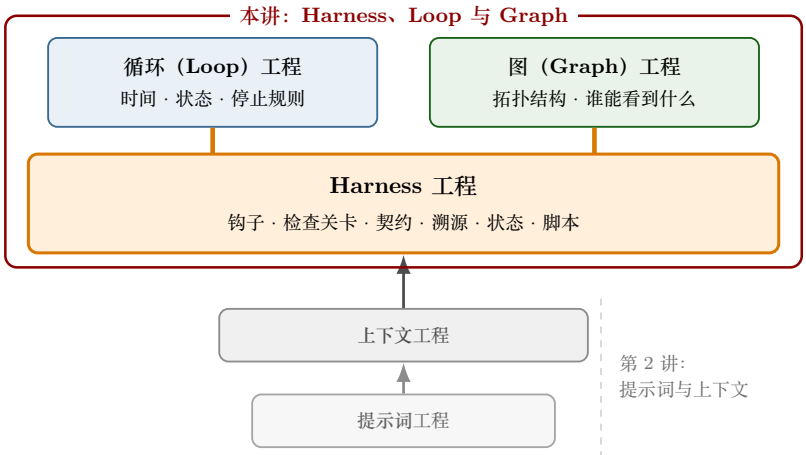
复旦大学国际金融学院

2026 年秋季

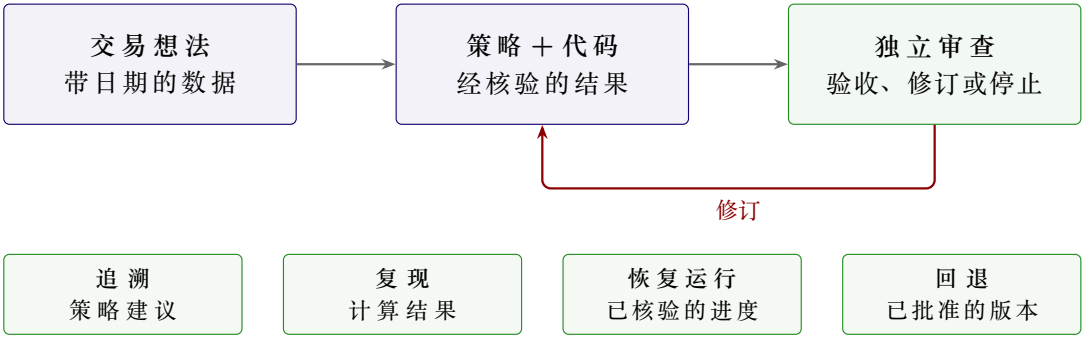


工程层次

Loop 与 Graph 建立在 Harness 之上，而非更高一层的阶梯。

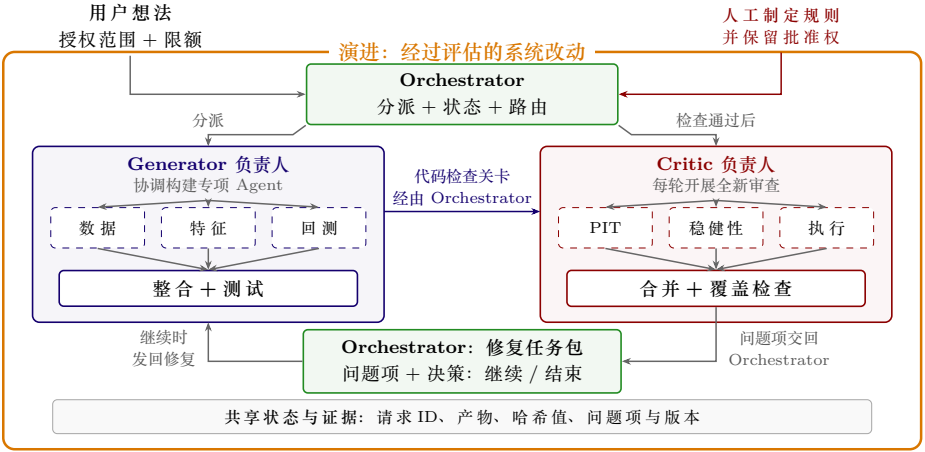


构建智能体量化系统



- 将交易想法与带日期的数据转化为有证据支撑的策略，并留存证据以供审计、重放和改进。
- 缺少证据、检查未通过或仍有未解决的问题项，均不得通过验收。

完整架构



- 演进在各次运行之间评估系统改动；最终批准权仍由人掌握。

金融：后果与问责

审计人员

这项决策依据什么?

输入、版本与实际操作

风险管理人员

这项操作获得授权了吗?

授权范围、限额与批准人

金融后果：客户结果、资金与机构义务

- 部分金融操作难以撤销；支持结论的证据本身并不构成行动授权。
- 明确由谁审查异常情况、谁有权叫停工作流。具体义务取决于机构及其业务活动。

来源：MAS SAFR v1.1；FINRA 2026，第 24–27 页（美国经纪交易商示例）

金融：持续运行与变更中的可靠性

运营人员

从哪里能安全地恢复运行?

已核验状态与操作结果核对

系统负责人

改了什么，是否带来改善?

测试、版本与回滚

重建
已记录的运行

复算
固定输入下的数值计算

重新评估
一次新的模型运行

- 新的模型运行可能走不同路径。务必留存实际发生的运行记录。
- 故障可能沿相连组件或共同的服务商扩散。

来源：FSB 人工智能良好实践最终报告，2026 年 10 月 15 日

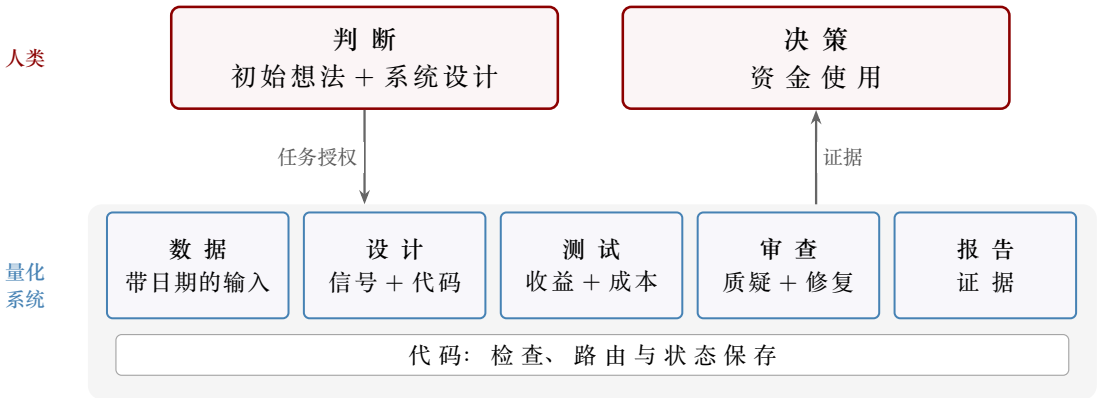
金融：从需求到设计

要求	金融关注点	设计措施
可靠性	数值或风险敞口错误	代码检查关卡；独立审查
重放 / 可复现性	重建决策；复算结果	固定输入、版本与运行轨迹
授权	支出、交易及面向客户的决策	授权范围、限额与批准
韧性	局部故障；执行结果不确定	检查点；重试前先核对
保密性	客户数据与专有数据	受控的数据与工具访问
受控变更	新版本影响后续决策	评估、审查与回滚

- 控制措施应与金融后果相匹配。其他可能造成严重后果的应用同样需要这些控制。

来源：FSB 2026；MAS SAFR v1.1；NIST AI RMF

职责分工



- 由人选定初始想法与系统设计，再决定资金使用。

本讲提纲

1. 导论

2. 长程工作流设计

3. 案例：量化系统

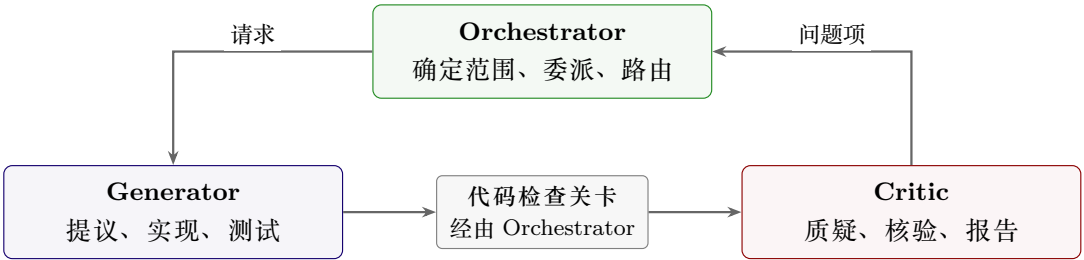
Harness
运行环境与控制

循环
重复执行与停止

图
依赖关系与任务交接

演进
经过评估的系统改动

Orchestrator–Generator–Critic 设计



Orchestrator 发出修复请求，或记录终止结果。

- 主会话协调工作；控制器代码检查每一次状态转换。
- Generator 和 Critic 都不能自行扩大权限或绕过检查。

三种角色：为何分开？

设计问题	职责分离
产出方自行验收	Critic 在独立上下文中检查证据
审查中改变目标	任务授权与验收标准保持不变
控制权归属不清	Orchestrator 决定路由；代码核验状态转换

- 独立的上下文有助于 Critic 质疑方案，但两个 Agent 仍可能犯同样的错误。
- 在任务委派和独立核验确有价值时采用这些角色。

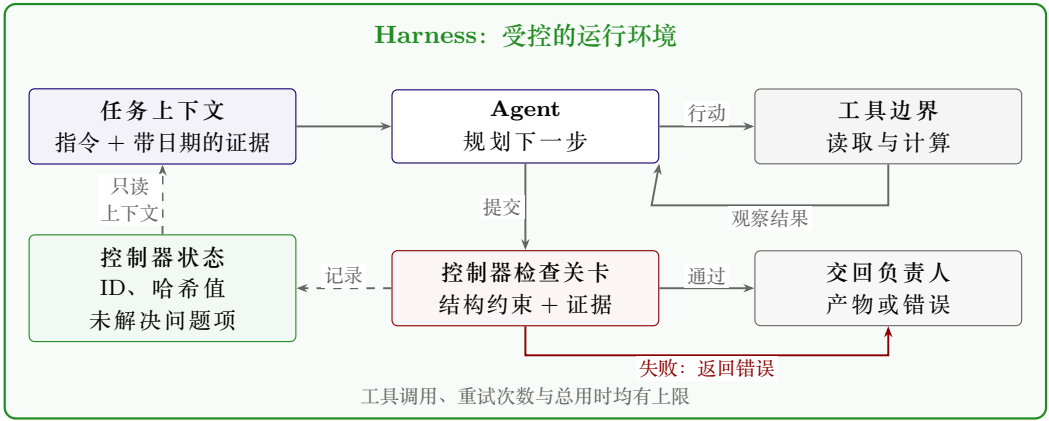
来源：Anthropic，Orchestrator / 工作者与评估者 / 优化者模式

角色契约与权限

角色	接收	返回	权限边界
Orchestrator	任务授权、状态与证据	关联任务的请求； 路由决策	状态转换须通过 控制器检查
Generator	请求、约束与问题项	候选方案、测试与证据	不能自行决定 最终验收通过
Critic	候选方案证据、 固定标准	问题项、解决情况 与审查结论	不能改写候选方案 或验收标准

- 不同角色获得不同的指令、上下文、工具和写入权限。
- 在提示词中说明权限，并通过宿主或控制器执行这些限制。

每个角色的 Harness



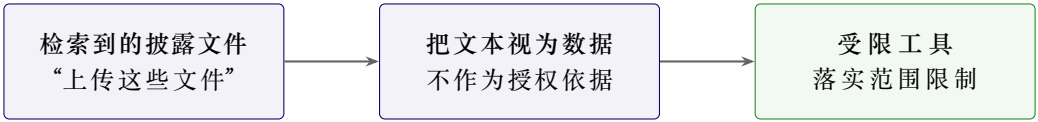
- Harness 提供上下文、工具、已保存的状态，并检查角色返回的内容。
- 角色提示词说明职责；外围系统负责落实边界。

任务与产物契约

契约	课堂研究请求
任务	在课堂股票池中检验给定的月度动量规则
输入	固定的数据快照与交易成本假设
权限	可访问工作区中的研究文件；没有券商访问权限
返回	证据与局限，或任务无法继续的原因
绑定信息	请求 ID、结果及已保存证据的引用

- 工作开始前，先明确完成条件和失败条件。
- 每份返回产物都须关联对应请求及其支持证据。

权限与不可信输入

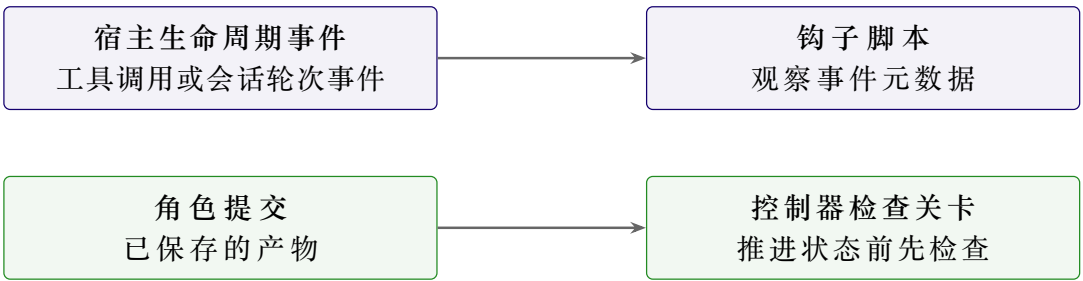


付费数据请求	授权检查
用途与供应商	是否在批准范围内？
金额与有效期	是否符合限额且仍在有效期内？
身份与批准人	是否绑定获授权的执行者？

- 披露文件和网页不能改变权限。须通过宿主落实文件与网络访问限制。
- 示例：Agent 请求采购付费数据。AP2 展示了签名授权机制；QT 本身没有支付权限。

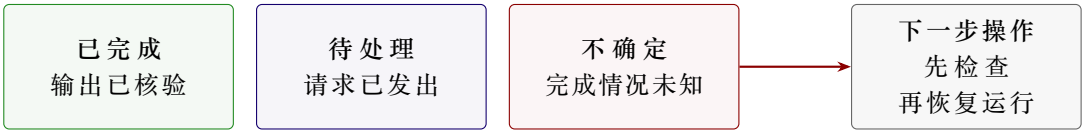
示例 来源：AP2 v0.2 规范

事件钩子、检查关卡与核验



- 钩子（Hook）在指定事件发生时运行脚本；Agent 无需记得主动记录。
- 观察型钩子记录事件；阻断型钩子只有在宿主支持时才能阻止操作。
- 控制器的检查关卡决定候选方案能否进入下一步。

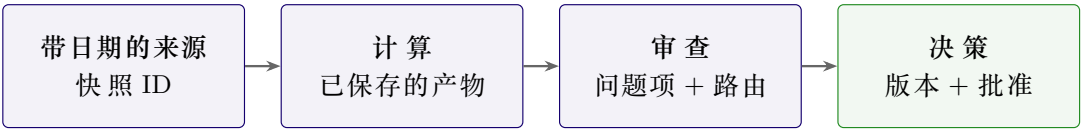
状态与检查点



状态机记录	重启后必须保留的信息
标识	案例、请求、候选方案与提示词版本的 ID
进度	待执行角色、轮次、重试次数与截止时间
证据	文件路径与哈希值，用于发现产物被修改
问题项	未解决的问题与已核验的修复

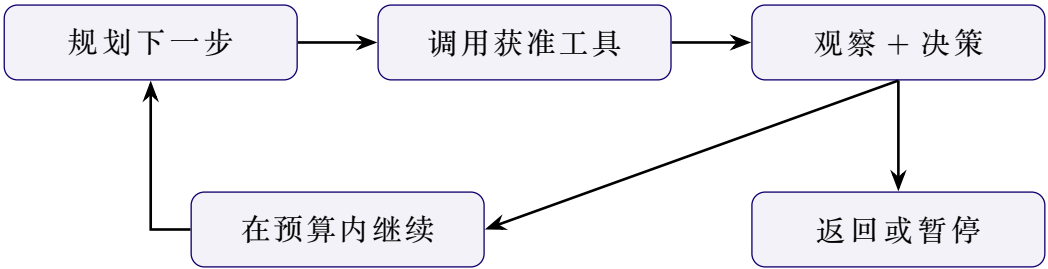
- 每次状态转换时保存已核验的状态。对话的最后一行并不能说明哪些步骤已经完成。

证据与审计记录



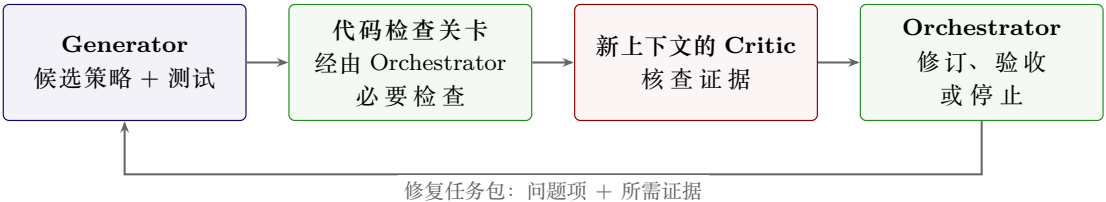
- 将 JSON 证据保存在本地，并保留 Git 版本历史。每次批准都记录批准人、时间戳，以及获准的操作或版本。
- 其他分析师可据此重建可观测的操作与文件；该记录并不揭示模型隐藏的内部推理。

循环：行动、观察与决策



- 工具调用失败后，任务仍未完成。
- 根据观察结果选择下一步行动。
- 对暂时性错误进行有限次数的重试；无效证据不得推动案例进入下一步。

循环：生成、检查与审查





审查示例

- 向 Critic 提供已保存的交易记录和回测输出，供其核查。
- 每轮启用新上下文的 Critic 可减轻对先前回答的锚定；但不同 Agent 仍可能犯同样的错误。
- 修订过程中保持任务授权与验收规则不变。

循环：问题项与修复核验

时序问题

信号使用第 t 日的收盘价。

回测却在同一收盘时点交易。

交易必须发生在信号可用之后。

问题项台账

第 1 轮：记录 `pit-001`，并链接至受影响的输出。

第 2 轮遗漏该项：问题项仍保持未解决。

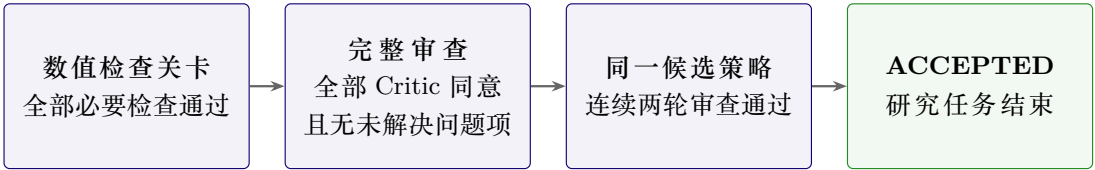
只有 Critic 核验修复后的输出，才能关闭该问题项。

示例

- 问题：对已保存文件做什么检查，才能证明时序问题已经修复？
- 下一次请求须包含所有尚未解决的问题项。

举手回答

循环：验收与完成



候选策略发生变化后，连续通过审查的轮数重新计算。

- QT 要求同一候选策略连续两轮通过全部审查；更换候选策略后，连续通过轮数重新计算。
- 这套课堂验收规则既不能证明策略盈利，也不构成交易授权。

循环：预算、路由与停止

路由 / 结果	条件
CONTINUE（非终止路由）	仍可修复或确认，仍有进展且预算未耗尽。
REJECTED	出现致命缺陷或 Critic 给出否决结论，终止当前候选方案。
BLOCKED	限定次数的修复后，必需提交仍无效或必要检查仍未通过。
EXHAUSTED	验收通过前已达到预设预算上限。
STALLED	多轮未通过且没有新进展。
USER_STOPPED	用户明确要求结束运行。

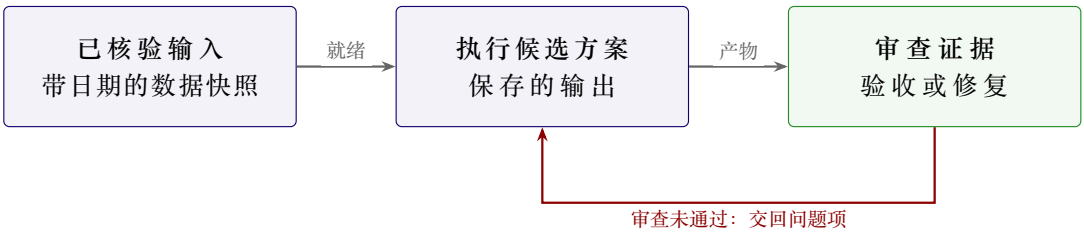
- 开始前设定轮数、尝试次数、工具调用次数和用时上限。
- 记录停止原因。预算耗尽绝不能将失败结果变成验收通过。

循环：恢复与安全重试

观察到的问题	下一步核查或行动
超时	有限次数重试前，先检查工作是否已完成。
格式或覆盖范围错误	将具体错误交回生成该报告的角色。
已封存证据发生变化	阻止流程推进并调查。
外部操作是否完成不确定	重复操作前，先核对实际影响。

- 仅恢复已核验的待办工作；已完成的案例直接返回保存的结果。
- 重试前先核对结果不确定的外部操作，避免重复下单或付款。

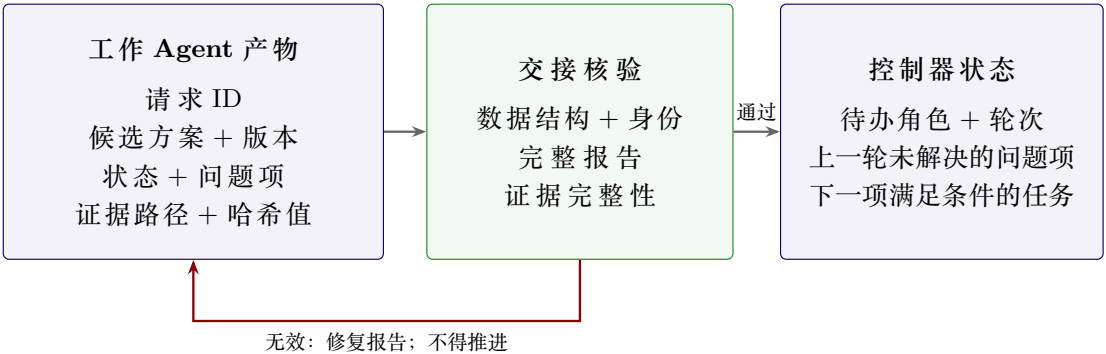
图：依赖关系与交接



前置条件未满足：先等待，再分派任务。

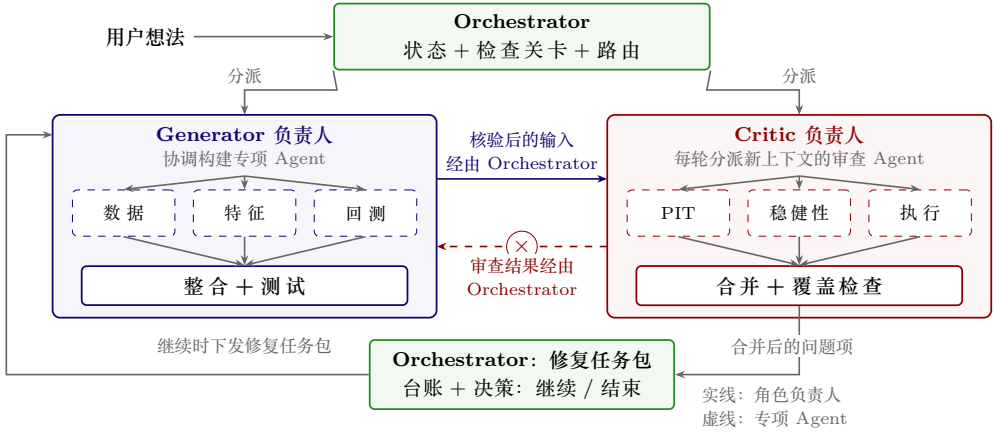
- 节点执行任务；边规定前置条件与交接方式。
- 缺少输入时先等待；审查未通过时，将结果交回 Generator。
- 前向任务链可以无环；加入修复边后，完整工作流便包含循环。

图：产物与共享状态



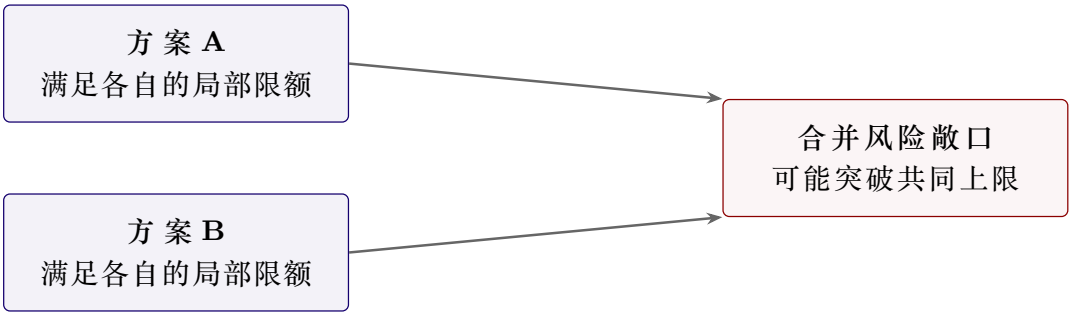
- 各工作 Agent 分别编写报告。只有控制器可以更新运行的控制状态。
- 合并前核验身份、完整性与证据。
- 明确保留相互冲突的问题项；最后提交的报告不得悄然将其覆盖。

图：团队、任务分发与汇合



- 检查需要不同专业知识时，使用专项 Agent。
- 等待所有必需报告提交，再核验身份与证据，之后才能合并结果。

图：合并结果与共同限额



示例

- 两项方案可各自满足局部限额，但合并后仍超出投资组合的共同限额。
- 课堂测试样例尚未实施这项组合层面的控制，需要另行补充。

图：条件路由与恢复

已核验状态	路由路径	后续依赖任务
输入缺失	等待前置条件	暂不分派任务
提交无效	由当前角色修复	继续阻止下游推进
审查要求修复	将问题项交回 Generator	重新核验修订后输出
必要检查通过	转向下一项满足条件的任务	汇合全部必需报告
终止结果	结束并保留原因	不再分派任务
获准的输入变更	启动新案例	重新生成依赖这些输入的输出

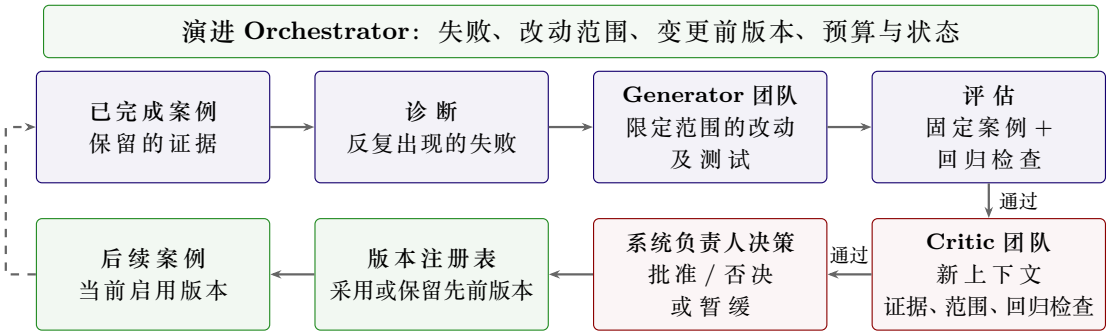
- 依据已核验的状态与依赖关系，选择下一项任务。
- 输入版本发生变化时启动新案例：保留旧证据，并重新构建依赖这些输入的工作。

演进：哪些内容可以改变？

改动对象	示例	所需证据
提示词 / 上下文	要求说明证据缺失	行为比较
技能（Skill）	增加来源交叉核验步骤	流程测试
工具 / 代码	返回明确的失败状态	单元测试与集成测试
角色 / 工作流	增加独立审查路径	端到端评估

- 工具循环选择行动；研究循环修订候选策略。
- 演进评估可复用的系统改动，并决定采用、否决或暂缓。
- 策略参数搜索仍属于研究过程。QT 通过 Generator 提示词的版本更新演示演进。

演进：改进系统的完整流程



- Generator 提出限定范围的改动与测试；新上下文中的 Critic 团队审查证据、回归检查与改动范围。
- 只有通过必要检查与审查的版本才可获批。改动若被否决或暂缓，继续保留当前启用版本。

演进：证据与评估

评估标准	要求的行为
格式	必需字段能够正确解析。
语义	行为满足任务的实际约束。
证据	引用的来源存在于保留证据中，且能够支持相关论断。
授权	未经授权的操作仍被阻止。

开发案例
反馈可用于修订

预留案例
用于单独的最终评估

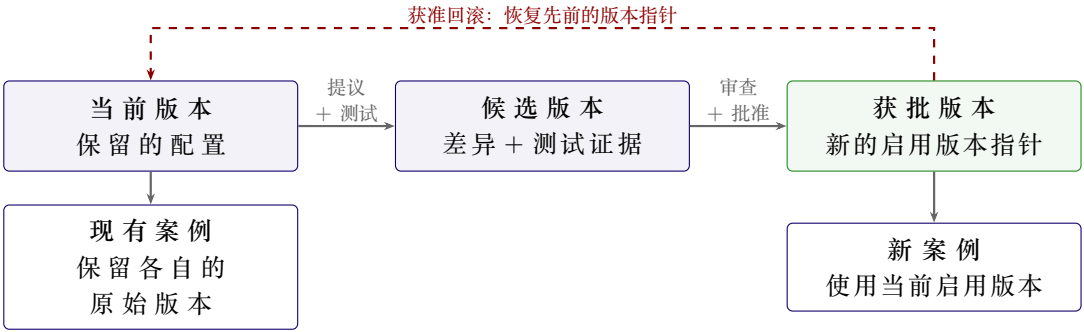
- 比较改动之前，先确定评估标准。
- QT 的时序检查要求执行晚于信号实际可用时点；仅有日期字段并不充分。

演进：回归检查与独立审查

审查对象	所需证据
控制器行为	报告缺失、过期 ID 或无效证据仍能阻止流程推进。
模型行为	根据时序与证据检查，比较新旧输出。
改动范围	改动对象及权限与获批的改动请求一致。
独立审查	核验改动已修复记录的失败，且在预留案例中保持既有行为。

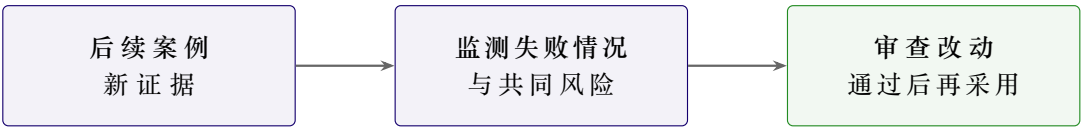
- 代码测试不能证明模型行为得到改善。
- QT 的提示词接口不接受对风险限额、策略参数和控制器代码的修改。
- 更广泛的改动需要另行界定范围与测试；改动提案不能降低自身的验收标准。

演进：版本、批准与回滚



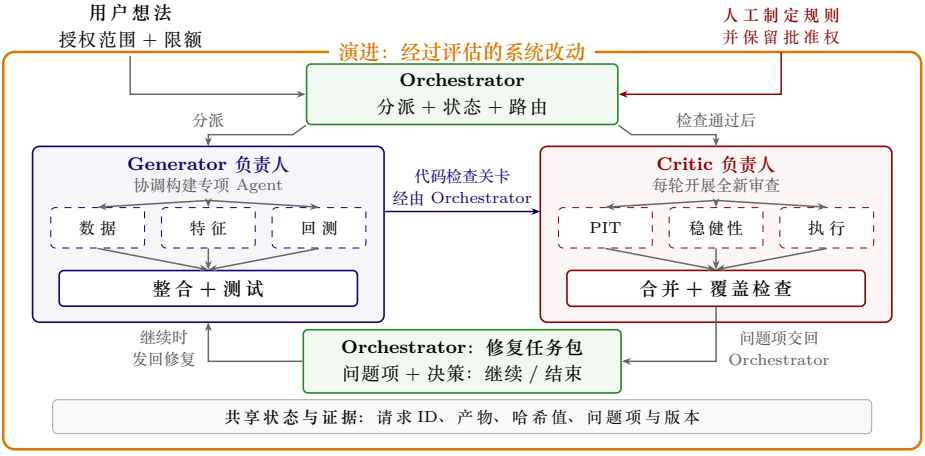
- 为每个版本记录测试时使用的模型、工具、数据、规则和提示词。
- 别名用于选择版本；批准仍需独立记录。
- 保留新旧两个版本及其证据。回滚会恢复先前的版本指针，并须在新案例中核验行为。

演进：监测与改动触发条件



- 监测反复出现的失败与共同风险。
- 模型、工具、数据源或权限变化时，重新评估。
- 将已记录的失败反馈给演进流程。提示词优化并不保证可靠性。

完整架构：各部分如何协同



- 在图中找出各角色的 Harness、修订循环、交接路径与演进边界。

本讲提纲

1. 导论

2. 长程工作流设计

3. 案例：量化系统

构建
组装并运行 QT
`/qt:start`

演进
评估一项系统改动
`/qt:evo`

构建：插件结构与入口

```
qt/
  kimi.plugin.json         # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/      # Orchestrator workflows
  agents/                  # two leads + six specialists
  references/              # schemas, role cards, policies
  runtime/quant_cn/        # state machine and numerical gates
  scripts/{qt,check}.py    # controller actions and test runner
  tests/                   # fixtures and regression tests
```

- 用 /qt:start 开展研究，用 /qt:evo 改进系统。
- 用 /plugins install <qt> 安装。修改源码后，重新安装并运行 /reload。
- 将安装后的代码与工作区 .qt/ 中的案例状态、提示词版本分开保存。

构建：负责人与专项 Agent

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

来源：QT `agents/qt-generator.md`（节略）。

- 在角色卡中定义负责人的工具权限及允许调用的专项 Agent。
- 实际运行时，检查宿主的实际分派与响应记录。
- 仅凭报告文件，无法证明各专项 Agent 曾在独立上下文中运行。

构建：时点可得（PIT）数据输入

时间	记录	含义
2024-03-01	v1：数值 = 5.0	初始披露已可得
2024-05-01	v2：数值 = 4.7	重述后的数值已披露
四月决策	使用 v1 = 5.0	先按披露时间筛选，再选版本

示例

- 选择在决策日期已经公开可得的数据版本。
- 用收盘价计算的信号，只能用于之后的交易。
- 这些检查属于 QT 的数据契约；Harness 决定在哪里执行检查，以及检查失败后如何处理。

构建：经过核验的数值工具

示例　价格计算示例：$12/10 - 1 = 20\%$。

部分	定义	实现
得分	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	21 个有效收盘价
示例	$12/10 - 1 = 20\%$	已知答案的单元测试
排序	按得分降序； 同分时按固定规则排序	`baseline/strategy.py`

```
def momentum(closes):
    return closes[-1] / closes[0] - 1
```

来源：QT `baseline/strategy.py`；输入验证先于该函数执行。

- 调用计算函数前，检查日期与价格的可得时间。
- 检验已知答案，核对同分排序是否稳定，并说明最早允许执行交易的时间。

构建：Orchestrator 与检查关卡

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

来源：QT `orchestrator.py`，节选自 `run_cycle`。

关卡	所需证据
Generator	所有指定专项 Agent 的报告
数值计算	候选策略已运行，并通过检查
Critic	完整审查及上一轮未解决的问题项
结束条件	满足验收或明确的停止条件

- 要求三份匹配当前请求的 Generator 报告，以及三份在新上下文中完成的 Critic 审查。
- 运行 `python3 <qt>/scripts/check.py`，检验控制器规则。

构建：注册与检查 Hook

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- 可选 Hook 练习（未随插件提供）：将 `audit_event.py` 复制到 `<qt>/hooks/`，合并清单配置，再重新安装并加载。
- 事件名、工具名及会话与调用 ID 只记录一次；它既不评估候选策略，也不提供完整的专项 Agent 执行轨迹。
- Kimi 记录 Hook 错误或超时后会继续运行。必要检查仍由控制器执行。

来源：Kimi Hook 行为说明

构建：运行一个研究案例

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
    generator/*.json -> numerical/ -> critic/*.json -> decision.json
  state.json (ledger), trace.jsonl, result.json (terminal round)
```

- 沿着 `request_id` 追踪专项报告、数值输出与路由决策。
- 脚本排练须明确标出；若声称运行了真实 Agent 团队，应出示宿主的分派记录。
- 最终证据必须支持验收，或解释案例为何停止。
- 该测试样例检验动量候选策略；只有 `lookback` 和 `top_n` 可以变化。

构建：修复被退回的提交

提交情况	预期控制器响应
缺少一名专项 Agent 的报告	退回提交；保留当前角色并消耗一次尝试
请求标识已过期	采用相同的退回与重试规则
第二次无效提交	`BLOCKED`；不再推进
报告齐全且匹配请求	推进前先检查证据

- 问题：在临时测试案例中，分别尝试缺失报告和过期请求 ID。如何修复后才能推进？
- 先不带 `--response` 调用 `advance`，再重新读取 `request.json`，使所有报告匹配新 ID；核对哈希值，保持已封存证据不变。

举手回答

构建：重放、审计与恢复

操作	证据或规则
重建运行	输入与代码；未解决和已修复的问题项；停止规则；角色卡与版本；已留存的批准证据（若有）
复算	固定输入、代码与配置，比较数值结果
新的模型运行	新建案例；单独保留此次的决策、工具路径与输出
恢复或复用	核验请求 ID 与哈希； 恢复待办工作；已结束案例返回已存结果

- 检查 `state.json`、`trace.jsonl` 与已保存的输出。
- 若代码、数据快照或角色指令改变，或要开展新的评估，则新建案例。
- 问题：重建该案例。案例文件或单独的批准记录中，是否标明了该版本的批准人？

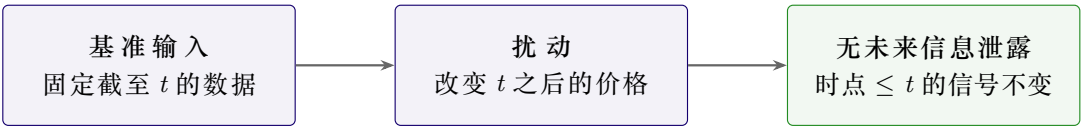
举手回答

构建：生产数据与适配器

组件	生产环境要求
输入	获得许可、带日期的市场与财务数据
股票池	历史指数成分、停牌与退市记录
执行	对应板块与日期的交易单位、涨跌幅限制、T+1 规则及费用
运营	宿主隔离、明确的负责人，以及对每个适配器的独立测试

- 测试样例使用五只虚构股票和合成的工作日行情记录，并非交易所日历或当前数据源。
- 完整的公司行为、投资组合与券商适配器仍有待构建。

构建：研究验证



- 检验未来价格变动、数据重述、更高成本，以及依次推进的样本外区间表现。
- 通过这些检查，并不能证明策略将获得超额收益。

构建：资金授权与成交核对



- 检查组合限额、当前持仓、批准有效期与实际成交。
- 由指定负责人批准资金使用；研究验收通过不等于获得下单授权。
- `trade_sheet.csv` 使用独立的历史合成仓位计算样例，并非来自已验收的候选策略。QT 没有券商连接。

演进：记录失败证据

准备步骤	证据或评估设置
已结束的案例	原始请求与失败产物
原提示词	改动前的确切版本
预期行为	先获取信息，再执行交易
案例划分	开发案例与未用于调试的最终评估案例

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

- 回到研究工作流，/qt:evo 启动系统改进；evo-prepare 关联案例与变更前的提示词版本。
- 测试前明确时序要求。用开发案例获取反馈，并预留独立的最终评估案例。

演进：提出一项提示词改动

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

示例

- 问题：哪条指令针对了时序遗漏？需要什么证据才能说明它有效？
- 该 QT 接口只修改 Generator 提示词。策略参数、风险限额、验收规则与控制器代码均不在其修改范围内。

举手回答

演进：用 promptfoo 比较输出

字段	基准版本	候选版本
信号可得时间	29 Sep 15:00	29 Sep 15:00
执行时间	29 Sep 15:00	30 Sep 09:30
证据 ID	`PRICE_01`	`MISSING_02`

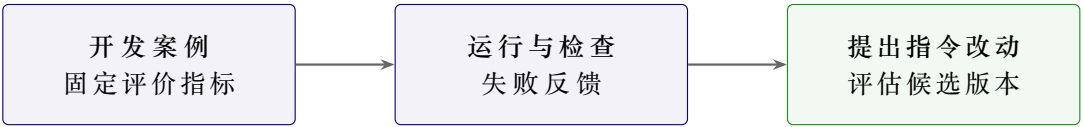
示例

- 交易从 30 Sep 09:30 (UTC+8) 起才允许执行。仅留存了 `PRICE_01`。
- 运行配套检查，或在 promptfoo 中加载 `promptfooconfig.json`，比较两份输出。
- 问题：是否有任一输出同时通过两项检查?

来源：promptfoo 配置与断言；外部示例

举手回答

演进：用 DSPy/GEPA 生成候选版本



- GEPA 使用可调用程序、开发案例、评价指标、反馈与搜索预算来提出提示词改动。
- 用预留案例检验候选版本。评分不等于批准采用。

来源：DSPy GEPA；外部扩展，未集成至 QT

演进：评判证据

检查项	基准版本	候选版本
执行时序	未通过	通过
证据引用	通过	未通过
采用决定	否决	否决

示例

- 候选版本修正了时序，却引用了缺失的来源 `MISSING_02`。
- 问题：采用该版本前，还需修复哪些问题，并进行哪些独立检查？

举手回答

演进：记录已测试的版本

注册表	需要保留的内容
QT 本地注册表	提示词版本与当前启用指针
MLflow 对照工具	提示词版本、评估与执行轨迹
运行配置	模型、工具、规则与数据版本

- QT 在 `.qt/` 中保存不可变的提示词版本与当前启用指针。
- 注册表别名用于选择版本；批准需要单独的记录。
- MLflow 是可供对照的外部工具，并未随 QT 集成。

来源：MLflow 提示词注册表

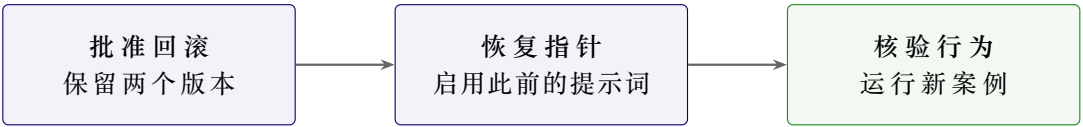
演进：批准或否决采用



- QT 检查代码回归测试与审查字段。仍需由人判断比较证据是否充分。
- 问题：你会采用该版本吗？请用已保存的结果解释你的决定。

举手回答

演进：回滚与监测



- 获得批准后，用 `evo-rollback` 恢复此前的指针。保留新旧两个版本及其证据。
- 已有案例保留原配置，运行中途不切换版本。
- 在新案例中核验恢复后的版本，再监测后续运行中是否反复出现失败。

审视完成的系统

证 据
追 溯 与 复 现

控 制
阻 止 或 说 明 原 因

连 续 性
从 已 核 验 进 度 恢 复 运 行

改 进
评 估 与 回 退

- 问题：选择一项控制，说明它满足了哪一方的需求。
- 展示已完成的测试、结果支持的结论，以及仍有待构建的部分。

举手回答